

Go-Zero IM 项目企业级升级路线文档

一、项目升级目标

当前项目目标不是继续堆功能，而是将现有 Go-Zero IM 项目从“能聊天的 Demo”升级为“具备企业级工程深度的分布式实时消息投递系统”。

升级后的项目应重点体现以下能力：

1. 高并发 WebSocket 长连接管理
2. 多 Gateway 实例下的用户在线路由
3. 单聊消息可靠投递
4. 会话级消息顺序保证
5. ACK、pending、断线重连补偿机制
6. Kafka 群聊异步扩散
7. Redis 在线状态与 TTL 续期
8. 消息幂等与重复消费处理
9. 结构化日志、traceld、链路追踪
10. WebSocket 压测与性能复盘

项目最终定位：

基于 Go-Zero 的分布式 IM 与实时消息投递系统

未来可复用为 AI 产品中的实时事件通知底座，例如任务状态推送、Agent 执行进度通知、系统消息推送等。

二、总体版本规划

建议分为 6 个版本逐步升级：

- V0：现状梳理与基线整理
- V1：消息模型与单聊可靠投递升级
- V2：多 Gateway 在线路由与长连接治理
- V3：断线重连与离线消息补偿
- V4：Kafka 群聊异步扩散与幂等消费
- V5：可观测性、压测与工程化完善
- V6：实时事件通知扩展，为 AI 产品复用预留能力

其中：

- V1 - V3 是秋招核心竞争力
- V4 - V5 是企业级项目深度
- V6 是未来 AI 产品复用方向，可选

V0：现状梳理与基线整理

目标

先不要急着重构代码。第一步是让 Codex 先理解当前项目结构，明确已有能力、缺失能力和升级风险。

需要 Codex 检查的内容

1. 当前项目目录结构
2. 当前是否已有 Gateway / Logic / Transfer 服务
3. WebSocket 连接建立流程
4. 用户登录与鉴权流程
5. 当前消息发送链路
6. 当前 Redis 使用情况
7. 当前 Kafka 使用情况
8. 当前 MySQL 表结构
9. 当前 gRPC / Etcd 服务发现情况
10. 当前是否已有 ACK、seq、离线消息、重连补偿

产出

生成一份 `docs/upgrade_assessment.md`，内容包括：

1. 当前项目架构
 2. 当前已有能力
 3. 当前缺失能力
 4. 当前代码耦合点
 5. 推荐升级顺序
 6. 每个版本涉及的文件和模块

验收标准

Codex 能输出当前项目的真实结构分析，而不是直接开始乱改代码。

给 Codex 的任务描述

请先不要修改代码。请阅读当前 Go-Zero IM 项目结构，分析 Gateway、Logic、Transfer、Redis、Kafka、MySQL、gRPC、Etcd 等模块的现状，生成一份 `docs/upgrade_assessment.md`，说明当前项目已具备哪些能力、缺失哪些企业级 IM 能力，以及推荐的分阶段升级方案。

V1：消息模型与单聊可靠投递升级

目标

将当前“发一条消息”升级为具备可靠性基础的消息模型。

重点解决：

客户端重复发送怎么办？
服务端如何生成唯一消息？
同一会话内消息如何排序？
消息是否落库？
投递失败怎么记录？

新增或调整核心字段

消息表建议包含：

```
id
server_msg_id
client_msg_id
conversation_id
conversation_type
from_user_id
to_user_id
group_id
seq
content
content_type
status
created_at
updated_at
```

其中：

client_msg_id: 客户端生成，用于防重复提交
server_msg_id: 服务端生成，全局唯一消息 ID
conversation_id: 会话 ID
seq: 会话内递增序号
status: 消息状态，例如 sent / delivered / read

关键设计

1. clientMsgId 幂等

客户端每次发送消息时携带 `clientMsgId`。

服务端建立唯一约束：

```
from_user_id + client_msg_id
```

如果客户端超时重试，服务端发现该 `clientMsgId` 已存在，则不重复落库，而是返回已有消息结果。

2. 会话级 seq

系统不保证全局消息顺序，只保证单会话内顺序。

每个 conversationId 维护递增 seq：

```
conversation_id = 1001  
seq = 1, 2, 3, 4, 5...
```

客户端按 seq 展示消息。

3. 单聊基本链路

```
客户端发送消息  
↓  
Gateway 接收 WebSocket 消息  
↓  
Gateway 转发给 Logic  
↓  
Logic 校验 clientMsgId  
↓  
Logic 生成 serverMsgId  
↓  
Logic 生成 conversationSeq  
↓  
消息落库  
↓  
查询接收方在线状态  
↓  
在线则实时投递  
↓  
不在线则等待离线补偿
```

需要修改的模块

im-gateway: 接收消息协议增加 clientMsgId
im-logic: 新增消息幂等、seq 生成、消息落库逻辑
model: 新增或调整 message 表
common: 新增 messageId、traceId、错误码工具

产出

1. 新 message 表结构
2. clientMsgId 幂等逻辑
3. serverMsgId 生成逻辑
4. conversationSeq 生成逻辑
5. 单聊消息落库链路
6. 单聊发送接口或 WebSocket 消息协议更新

验收标准

1. 同一个用户使用同一个 clientMsgId 重复发送，不会产生两条消息。
2. 同一会话内消息 seq 连续递增。
3. 消息发送后能落库。
4. 消息返回中包含 serverMsgId 和 seq。
5. 代码中有清晰的错误处理和日志。

面试价值

这一版可以支撑以下面试问题：

消息重复发送怎么办？
IM 系统怎么保证消息顺序？
为什么只保证单会话顺序？
clientMsgId 和 serverMsgId 分别有什么作用？
消息落库在投递前还是投递后？

V2：多 Gateway 在线路由与长连接治理

目标

将项目从单 Gateway 聊天升级为多 Gateway 分布式长连接架构。

重点解决：

用户连接在哪台 Gateway?
Logic 怎么找到用户所在 Gateway?
Gateway 异常宕机后在线状态怎么失效?
心跳怎么维护?

核心设计

用户建立 WebSocket 后，Gateway 在 Redis 中写入在线状态：

```
presence:{userId}:{deviceId} -> gatewayId:connId  
TTL = 60s
```

心跳续期：

客户端每 30s 发送心跳
Gateway 收到心跳后续期 Redis key

断开连接：

WebSocket 正常断开时删除 presence key
Gateway 异常宕机时依赖 TTL 自动过期

Logic 投递消息时：

```
Logic 查询 Redis presence key  
↓  
如果在线，获取 gatewayId  
↓  
通过 gRPC 调用目标 Gateway 的 Push 接口  
↓  
Gateway 根据 connId 找到本地 WebSocket 连接  
↓  
投递消息
```

需要新增或调整的模块

```
im-gateway:  
- 连接管理器 connManager  
- userId/deviceId/connId 映射  
- 心跳处理  
- gRPC Push 接口  
  
im-logic:
```

- 查询 Redis 在线状态
- 调用目标 Gateway Push

Redis:

- presence key 设计
- TTL 续期

Gateway 本地连接管理

建议维护:

```
connId -> websocket connection  
userId -> connId list
```

如果考虑多端登录, 则使用:

```
userId + deviceId -> connId
```

产出

1. Gateway 连接管理器
2. Redis 在线状态注册
3. 心跳续期机制
4. Gateway gRPC Push 接口
5. Logic 根据 Redis 路由到指定 Gateway

验收标准

1. 用户连接 Gateway 后, Redis 中能看到 presence key。
2. 心跳正常时 TTL 能续期。
3. 用户断开后 presence key 被删除。
4. Gateway 异常退出时, presence key 能通过 TTL 自动过期。
5. 多 Gateway 启动时, Logic 能将消息投递到用户所在 Gateway。
6. 用户不在线时, Logic 不进行实时投递, 转入离线流程。

面试价值

这一版可以支撑以下面试问题:

```
多 Gateway 下怎么找到用户连接?  
为什么在线状态放 Redis?  
Redis key 为什么要设置 TTL?  
Gateway 宕机后在线状态怎么处理?  
Gateway 和 Logic 为什么要拆?
```

V3: ACK、pending、断线重连与离线补偿

目标

升级消息可靠性，不再依赖 WebSocket 本身的可靠性。

重点解决：

消息推送出去就一定成功了吗？
客户端没收到怎么办？
客户端断线期间的消息怎么补？
用户重连后怎么拉缺失消息？

核心设计

1. ACK 机制

服务端投递消息后，客户端收到消息需要返回 ACK：

ACK 内容：
serverMsgId
conversationId
seq
userId
deviceId

服务端收到 ACK 后更新投递状态。

2. pending 投递状态

新增投递表：

```
message_delivery
- id
- server_msg_id
- conversation_id
- seq
- user_id
- device_id
- status
- retry_count
- last_push_time
- ack_time
```



```
- created_at
- updated_at
```

状态:

```
pending
delivered
failed
```

3. 断线重连补偿

客户端本地维护:

```
conversationId -> lastSeq
```

重连时发送:

```
userId
deviceId
lastSeqMap
```

服务端查询:

```
select * from messages
where conversation_id = ?
and seq > ?
order by seq asc;
```

然后补发缺失消息。

需要修改的模块

```
im-gateway:
- 接收客户端 ACK
- 重连时接收 lastSeqMap
- 补发消息给客户端

im-logic:
- 生成 message_delivery
- 更新 ACK 状态
- 查询缺失消息
- 补偿消息
```

model:
- 新增 message_delivery 表

产出

1. 客户端 ACK 协议
2. message_delivery 表
3. pending 状态记录
4. ACK 后状态更新
5. 断线重连补偿接口
6. 根据 lastSeq 拉取缺失消息

验收标准

1. 服务端推送消息后生成 pending 记录。
2. 客户端 ACK 后，pending 更新为 delivered。
3. 客户端断开期间收到的消息，重连后能根据 lastSeq 补拉。
4. 重复 ACK 不影响状态正确性。
5. 同一条消息不会因为补偿导致重复落库。
6. 补偿消息按 seq 顺序返回。

面试价值

这一版可以支撑以下面试问题：

WebSocket 断了消息会不会丢？
ACK 机制怎么设计？
pending 表有什么作用？
断线重连怎么补消息？
离线消息和历史消息有什么区别？

V4：Kafka 群聊异步扩散与幂等消费

目标

将群聊消息从同步扩散升级为 Kafka 异步扩散。

重点解决：

群聊成员很多时，发送链路会不会变慢？
Kafka 重复消费怎么办？

如何保证同一群聊内消息顺序？
群成员离线怎么办？

核心设计

1. 群聊发送链路

```
客户端发送群聊消息
↓
Gateway 转发 Logic
↓
Logic 校验并落库
↓
Logic 写入 Kafka
↓
立即返回发送成功
↓
Transfer 消费 Kafka
↓
查询群成员
↓
批量生成 message_delivery
↓
在线成员实时投递
↓
离线成员等待补偿
```

2. Kafka key 设计

Producer 发送 Kafka 消息时使用：

```
key = conversationId
```

目的：

同一个 conversationId 的消息进入同一个 partition，
保证单会话内消息消费顺序。

3. 消费幂等

投递表建立唯一键：

```
server_msg_id + user_id + device_id
```

Transfer 重复消费时，如果 delivery 已存在，则跳过。

需要修改的模块

```
im-logic:
- 群聊消息落库
- 发送 Kafka event

im-transfer:
- 消费 Kafka
- 查询群成员
- 批量生成 delivery
- 调用在线投递

model:
- group_member 表检查
- message_delivery 唯一约束
```

Kafka 消息结构

```
{
  "eventType": "GROUP_MESSAGE_CREATED",
  "traceId": "xxx",
  "serverMsgId": "xxx",
  "conversationId": "xxx",
  "fromUserId": 1001,
  "createdAt": 1710000000
}
```

产出

1. 群聊消息 Kafka event
2. Transfer 消费逻辑
3. 群成员查询
4. 批量 delivery 生成
5. 重复消费幂等
6. 在线投递与离线补偿衔接

验收标准

1. 群聊消息发送后，发送方无需等待所有成员投递完成。
2. Kafka 中能看到群聊消息事件。
3. Transfer 能消费并扩散给群成员。
4. 同一群聊内消息按照 seq 顺序处理。
5. Kafka 重复消费不会产生重复 delivery。
6. 离线成员重连后能补偿群聊消息。

面试价值

这一版可以支撑以下面试问题：

群聊为什么要用 Kafka？
Kafka 怎么保证消息顺序？
Kafka 重复消费怎么办？
群聊大群怎么削峰？
同步扩散和异步扩散有什么区别？

V5：可观测性、压测与工程化完善

目标

让项目从“功能能跑”升级为“工程上可验证、可定位、可复盘”。

重点解决：

项目性能如何证明？
消息失败怎么排查？
链路耗时怎么看？
系统瓶颈在哪里？

核心能力

1. traceId 链路追踪

每条消息生成 traceId，并在以下链路中传递：

Gateway 接收
Logic 处理
MySQL 落库
Redis 查询
Kafka 发送
Transfer 消费
Gateway Push
客户端 ACK

日志统一包含：

traceId
serverMsgId
conversationId

```
fromUserId  
toUserId  
gatewayId  
costMs
```

2. 结构化日志

日志格式示例：

```
[SendMessage] traceId=xxx serverMsgId=xxx conversationId=1001 from=1 to=2 cost=12ms  
status=success
```

3. 核心指标

建议至少统计：

```
当前 WebSocket 连接数  
消息发送 QPS  
消息投递成功率  
ACK 成功率  
平均延迟  
P95 延迟  
Kafka 消费积压  
在线用户数  
重连补偿次数
```

4. 压测脚本

压测场景：

1. 1w WebSocket 长连接建立
2. 心跳稳定性测试
3. 单聊消息发送测试
4. 群聊消息扩散测试
5. 断线重连补偿测试

产出

1. traceId 工具
2. 结构化日志
3. WebSocket 压测脚本
4. 压测结果文档
5. 项目复盘文档

压测报告建议结构

生成 `docs/benchmark_report.md` :

1. 测试环境
2. 服务部署结构
3. 测试机器配置
4. Gateway 实例数量
5. WebSocket 连接数
6. 心跳间隔
7. 消息发送数量
8. 成功率
9. 平均延迟
10. P95 延迟
11. CPU / 内存占用
12. 发现的瓶颈
13. 优化方案

验收标准

1. 每条消息链路能通过 `traceld` 查到完整日志。
2. 能统计当前在线连接数。
3. 能输出消息成功率和延迟。
4. 能进行至少 1w 连接压测。
5. 能生成压测报告。
6. 能说明系统当前瓶颈。

面试价值

这一版可以支撑以下面试问题：

你项目压测过吗？
支持多少连接？
P95 延迟是多少？
出现消息失败怎么排查？
怎么定位 Gateway、Logic、Kafka 哪一层慢？

V6：实时事件通知扩展，为 AI 产品复用预留能力

目标

这一版不是当前秋招必须项，而是为未来 AI 产品复用做预留。

将 IM 项目抽象为：

实时消息投递系统 + 实时事件通知系统

核心思路

除了普通聊天消息，系统支持事件消息：

```
text
system_notice
task_event
agent_event
```

消息类型字段：

```
message_type
```

示例：

```
1 = text
2 = system_notice
3 = task_event
4 = agent_event
```

AI 产品可复用场景

未来 AI 产品中可以推送：

```
TaskCreated
TaskRunning
AgentThinking
ToolCalling
CodeGenerated
TestFailed
PatchApplied
TaskFinished
```

这些事件都可以复用 IM 的 WebSocket 投递能力。

产出

1. message_type 扩展
2. system_notice 消息类型
3. task_event 消息类型

4. 事件消息投递 demo
5. 文档说明：如何复用为 AI 产品事件通知底座

验收标准

1. 普通聊天消息不受影响。
2. 系统可以发送 system_notice。
3. 系统可以发送 task_event。
4. WebSocket 客户端能区分不同 message_type。
5. 文档说明未来 AI 产品如何复用该能力。

面试价值

这一版可以支撑以下表达：

该系统不仅可用于 IM 场景，也可以作为 AI Agent 产品中的实时任务状态通知底座，用于推送任务执行进度、工具调用结果和系统通知。

三、推荐实际执行顺序

第一阶段：6 月 11 日 - 6 月 23 日

当前中期 PPT 优先，不建议大规模改代码。

只做：

1. V0 项目评估
2. V1 设计文档
3. 数据表结构梳理
4. 简历项目升级表达草稿

目标：

不要把项目改崩，先明确怎么升级。

第二阶段：6 月 24 日 - 7 月中旬

华为实习刚开始，时间有限。

建议只做：

1. V1 消息模型升级
2. clientMsgId 幂等
3. conversationSeq
4. 单聊消息落库
5. 基础 ACK

目标：

把可靠单聊链路打通。

第三阶段：7月中旬 - 8月初

开始集中升级项目深度。

建议做：

1. V2 多 Gateway 在线路由
2. Redis presence key
3. TTL + 心跳续期
4. Gateway gRPC Push
5. Logic 路由投递

目标：

让项目具备真正的分布式长连接架构。

第四阶段：8月初 - 8月中旬

升级可靠性。

建议做：

1. V3 pending
2. message_delivery 表
3. 断线重连 lastSeq
4. 离线消息补偿

目标：

能回答消息丢失、重复、断线补偿问题。

第五阶段：8月中旬 - 8月底

升级企业级深度。

建议做：

1. V4 Kafka 群聊异步扩散
2. Transfer 消费
3. 重复消费幂等
4. V5 traceId
5. 压测脚本
6. benchmark_report.md

目标：

秋招前形成完整项目闭环。

第六阶段：秋招后或有余力时

做 V6：

1. task_event
2. system_notice
3. AI 产品事件通知 demo

目标：

为未来 AI 产品复用做准备。

四、最终简历表达版本

当 V1 - V5 完成后，简历可以这样写：

项目名称

基于 Go-Zero 的分布式 IM 与实时消息投递系统

技术栈

Go、Go-Zero、WebSocket、gRPC、Etcd、Redis、Kafka、MySQL、Docker

项目描述

面向高并发长连接与实时消息投递场景，设计并实现分布式 IM 后端系统。系统拆分 Gateway、Logic、Transfer 等核心服务，支持 WebSocket 长连接维护、用户在线状态管理、单聊实时投递、群聊异步扩散、离线消息补偿和断线重连恢复，可作为聊天系统、系统通知和 AI 任务事件推送的实时通信底座。

项目亮点

1. 拆分 Gateway、Logic、Transfer 服务，Gateway 负责 WebSocket 长连接、心跳检测和连接注册，Logic 负责消息路由与会话状态，Transfer 负责 Kafka 异步消费和群聊扩散，实现连接层、业务层和异步投递层解耦。
2. 基于 Redis 维护 userId 到 Gateway 实例的在线路由关系，采用 TTL + 心跳续期处理异常断连，Logic 查询在线状态后通过 gRPC 调用目标 Gateway 完成实时消息投递。
3. 设计 clientMsgId、serverMsgId 和会话级 seq 的消息模型，结合 ACK、pending 状态和断线重连补偿机制，解决客户端重试、网络抖动和连接断开导致的消息重复与消息丢失问题。
4. 针对群聊同步扩散链路过长的问题，引入 Kafka 构建异步扩散流程，Producer 以 conversationId 作为 key 保证单会话内顺序，Transfer 消费后批量生成投递任务并完成在线推送与离线补偿。
5. 构建 WebSocket 压测脚本，测试多 Gateway 场景下的连接数、消息成功率、平均延迟和 P95 延迟，并通过 traceId 与结构化日志追踪消息完整链路，提升问题定位效率。

五、给 Codex 的总评估提示词

可以直接把下面这段发给 Codex：

请基于当前 Go-Zero IM 项目，评估将其升级为“分布式 IM 与实时消息投递系统”的可行性。不要一次性重构全部代码，请先完成 V0 现状评估，并根据以下版本路线判断每个版本需要修改哪些模块、涉及哪些表结构、哪些功能已有、哪些功能缺失、每个版本的预计改动量和风险。

升级路线：

V0：现状梳理与基线整理

V1: 消息模型与单聊可靠投递升级, 包括 clientMsgId、serverMsgId、conversationSeq、消息落库、幂等处理

V2: 多 Gateway 在线路由与长连接治理, 包括 Redis presence key、TTL、心跳续期、Gateway gRPC Push

V3: ACK、pending、断线重连与离线消息补偿, 包括 message_delivery 表、ACK 状态更新、lastSeq 补偿

V4: Kafka 群聊异步扩散与幂等消费, 包括 conversationId 作为 Kafka key、Transfer 消费、群成员扩散、重复消费幂等

V5: 可观测性、压测与工程化完善, 包括 traceId、结构化日志、连接数统计、消息成功率、P95 延迟、压测报告

V6: 实时事件通知扩展, 为未来 AI 产品复用预留 task_event、agent_event、system_notice 等消息类型

请输出:

1. 当前项目已有能力
2. 当前项目缺失能力
3. 每个版本需要修改的模块
4. 每个版本涉及的数据表变化
5. 每个版本的实现难度
6. 每个版本的风险点
7. 推荐优先实现顺序
8. 第一阶段最小可落地改造方案

六、不要做的内容

当前阶段不建议做:

朋友圈
红包
文件上传
音视频
复杂前端
AI 智能体聊天
RAG
MCP
Eino
Minio
Elasticsearch
复杂推荐系统

原因:

这些功能会分散项目主线, 且对后端秋招主项目帮助不如可靠消息、多 Gateway 路由、Kafka 异步扩散、压测和可观测性。

当前项目升级主线必须保持为:

可靠消息投递 + 分布式长连接 + 异步扩散 + 离线补偿 + 工程可观测